Enabling Data Privacy in Blockchain

immediate

ABSTRACT

some text ...

PVLDB Reference Format:
. Enabling Data Privacy in Blockchain. *PVLDB*, (): 1539-1552, .
DOI:

1. INTRODUCTION

some text ...

2. MOTIVATION FOR DATA PRIVACY

In this section, we motivate the need for providing data privacy in a blockchain network by introducing the following three use-cases: (1) food traceability, (2) patient-centric health-care, and (3) identity documents and certificates management. Each of these use-cases solves a real-world problem using blockchain technology. However, due to the lack of support for data privacy in existing blockchain technologies, some sensitive information could be leaked. For achieving the widespread adoption of blockchain, it is necessary to introduce a mechanism to preserve the privacy of sensitive data.

2.1 Food Traceability

In September 2006, there was an outbreak of food-borne illness caused by Escherichia coli bacteria [?] in 26 U.S. states. This outbreak was linked to bagged spinach. By October 6, 2006, 199 people had been infected, including three people who died and 31 who suffered kidney failure. It took regulators two weeks to conduct the backtrace and determine the exact source of the outbreak. During those two weeks, many people got sick, and tons of good spinach was unnecessarily discarded—wrongfully wasted because we could not tell the good spinach from the bad [?]. An efficient food traceability mechanism is the key to reduce the backtrace time in such a scenario. Traceability is the ability to track and record the movement of food products throughout a supply chain, which includes growing, harvesting, packing,

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-nc-nd/4.0/. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. , No.
ISSN 2150-8097.
DOI:

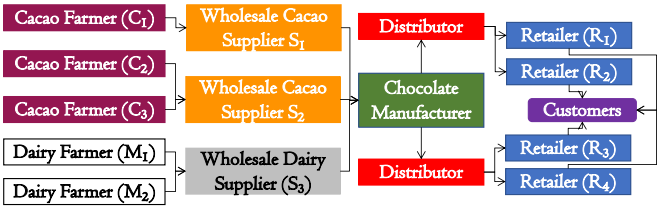


Figure 1: Actors involved in a simplified food supply chain.

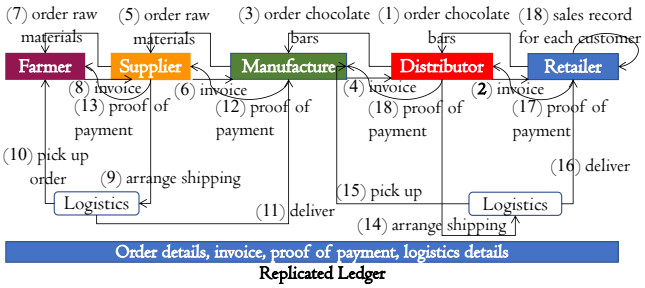


Figure 2: A sample transaction flow in a blockchain based chocolate supply chain network where all information pertaining to a transaction is visible in a replicated ledger.

manufacturing, repacking, storage, distribution, and sale to the consumer. With this information, it is possible to trace backward in the supply chain (usually to find the source of a problem) as well as trace forward (usually to do a recall of contaminated products). Food Standards Australia New Zealand (FSANZ) [?], European Commission's food safety policy [?], and Food and Agriculture Organization of United Nations (FAO) [?] provide various guidelines on what records need be kept in order to enable food traceability. The following are some of the records:

1. the name and address (and other contact details) of farmers/suppliers and a description of products.
2. the name and contact information of the logistics providers and a description of products transported.
3. the name and addresses (and other contact details) of customers and a description of the product supplied to them along with the product ID.
4. date of transaction or delivery.
5. batch or lot identification (or other markings such as product ID).
6. volume or quantity of product supplied or received.
7. raw materials used.

8. conservation and storage methods for the product.

As a result, when an outbreak of food-borne illness occurs, it would be possible to traceback from the customer → the product ID → the supplier/farmer. Similarly, a recall can be performed by tracing forward from the farmer/supplier → the product lot → customers. If these records are kept in the form of papers or stored digitally in a centralized database maintained at individual operators, during an outbreak, a lot of communication needs to happen between various food business operators to perform traceback or trace forward. Such a method is very inefficient, time-consuming, and expensive. Hence, a blockchain-based food traceability solution is needed for food safety. Further, such a solution helps to improve the overall supply chain efficiency, to allow us to easily communicate information that customers are asking for (i.e., fork to farm), to handle disputes quickly, and to help authenticate claims made in a product. With blockchain, network members can track the food products as they travel from farm to fork. Recently, Walmart, a retailer, did an experiment that traced the origin of sliced mangos from Walmart stores back to the farm. This process showed a radical improvement from approximately seven days it took to conduct the backtrace by using traditional methods down to 2.2 seconds [**?**, **?**] using Hyperledger Fabric.

Figure 1 shows actors involved in a simplified chocolate bars supply chain. For manufacturing a chocolate bar, cacao and milk are required (assume a bar of sugar-free chocolate). Usually, manufacturers buy raw materials from suppliers rather than directly contacting various farmers. Once chocolate bars are manufactured, it would be sent to various retailers across the globe via distributors so that retailers can sell the chocolate bars to customers. Note that distribution involves import/export authorities, customs, and other entities. For simplicity, we do not cover all involved actors.

Blockchain-Based Solution. Figure 2 shows a sample business process in a chocolate supply chain network. The process involves order placement, invoice creation, shipping arrangement, delivery, and proof of payment. We can implement the whole process using a permissioned blockchain platform with the help of smart-contracts. As each transaction is recorded in a replicated ledger, it enables the user to (a) perform the forward and backward tracing quickly and efficiently; (b) resolve disputes quickly; (c) maintain a tamper-evident record management for food traceability. Note that blockchain also helps in standardizing the business process and data format through smart-contract.

Data Privacy Violation. *Storing all information related to the above mentioned business process in a replicated ledger could leak proprietary or business competitive information. For example, invoice details cannot be shared as it could leak the business contract between the farmer and the supplier or the distributor and the retailer. As a result, a manufacture, a distributor and a retailer can find out the profit made by the supplier, manufacture and the distributor, respectively. This would result in unnecessary bargaining and conflicts between business partners.*

2.2 Patient-Centric Health-Care

The health-care domain faces the following challenges, which can be solved by using blockchain: (1) noninteroperable Electronic Medical Records (EMR) management system; (2) a lack of transparency and trust; (3) insurance fraud.

Noninteroperable EMR system. In health-care, a practitioner does not have access to all the historical medical records of a patient. This is because the Electronic Medical Record (EMR) systems assume that a patient sees only practitioners in one hospital. However, the reality is that, on average, a patient in the US sees 18.7 different practitioners/hospitals during his/her lifetime []. As existing EMR systems are hospital-centric, there is no interest to interoperate or share data with other hospitals due to competitions. As a result, the patient's EMR are trapped in different islands (on average 19 different locations []), which creates a hurdle to execute personalized health-care.

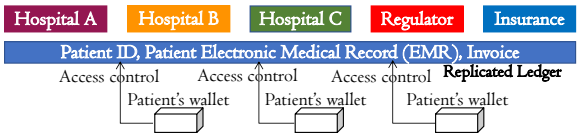


Figure 3: A blockchain based patient-centric health care management system where the health care records are visible in a replicated ledger and the read-write access is controlled by the patients.

Lack of Transparency & Trust. A hospital-centric delivery model using the current framework of existing technologies results in a lack of transparency and trust. A practitioner can provide a wrong treatment and later tamper the EMR to hide his mistakes, given that the EMR system is managed at his/her hospital. Moreover, the Health Insurance Portability and Accountability Act (HIPAA) states that patients should own their medical records and control access permission.

Insurance Frauds. There is a growing concern in the insurance industry on increasing number of health-care insurance frauds. The FBI estimates that health care fraud costs American taxpayers $80 billion a year []. The following are the commonly committed frauds. (1) duplicate and inflated bills; (2) manipulating pre-policy health check-up records; (3) inflated billing, billing for services not provided; (4) unwarranted procedures, excessive investigations, expensive medicines; (5) overutilization, extended length of stay; (6) fudging records, patient history.

Blockchain-Based Solution. To resolve the above mentioned problems, we need a blockchain based solution, as shown in Figure 3. Here, the patient owns their EMR and controls access permission using the blockchain wallet. Hospitals can create EMR for patients as a result of some medication or treatment. As the information is shared using a replicated and immutable ledger, it adds more transparency & trust, enables sharing of EMR records, maintains complete provenance and audit trails and reduces the number of fraudulent events in the insurance industries. Further, the smart-contract helps in standardizing the EMR data format so that sharing and processing of the data become simpler.

Data Privacy Violation. *Sharing of sensitive data such as EMR of a patient in a replicated ledger is usually unacceptable as it leaks private information about a patient. Further, it violates laws such as HIPAA privacy rule [], which dicdate the hospital to protect the patient's privacy.*

2.3 Identity and Diploma Certificate Management

Identity Frauds. In 2018, nearly 14.4 million people in the US were impacted by some form of identity theft according to a survey conducted by Javelin Strategy and Re-

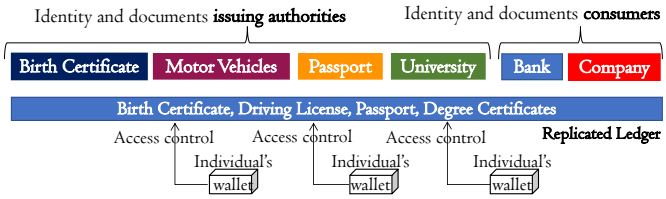


Figure 4: A blockchain based documents and certificate issuance system where all data are stored in an immutable replicated ledger and the read-write access is controlled by the individual who owns those data.

search []. Such frauds costed $1.7 billion in total for victims. The stoled identities were used to takeover existing accounts and open new accounts for loan and mortages, which results in a financial loss for the victims. According to interpol [], criminals make fake identities to carry out their illegal activities. Criminals use counterfeit and forged identity or travel document as well as genuine identity or travel document which was acquired using other counterfeit documents. Hence, there is a need to digitalize the identity and documents to avoid such frauds.

Certificate Frauds. Fradulent diploma certificates have become a global phenomemon due to the rising demand of qualification requirement in the job market []. Based on national data in US, 6% of bachelor degrees and 35% of associate degrees are fake cite. Due to lack of transparency and digitalization of degree certificates by universities, the companies need to trust the employee, and notraries. Hence, it is easily possible to produce and present fake diplomas.

Blockchain Based Solution. To tackle the abovem entioned problems, a blockchain-based solution can be used to issue certificates and to verify issued certificates. Figure 4 shows actors involved in such a blockchain solution. The issuing authorities store the documents such as birth certificates, passport, driving license, diploma certificates directly into the replicated ledger. These documents are owned by the respective individual and can control the access permission using the blockchain wallet. The consumers of documents and certificates can get the permission from the individual and verify the document for various purpose such as job appointment.

Data Privacy Violation. *Storing all sensitive document such as passport, diploma certificates in a replicated ledger could leak a lot of personal information about the individual. Hence, many of the individuals would not prefer to participate in such a blockchain network.*

3. PROPOSING PRIVATE DATA COLLECTIONS FOR DATA PRIVACY

Key Idea. To hide the sensitive data from specific members in a blockchain network, we can share the data only with the members who should see it and store the hash of data on the replicated ledger as evidence for tamper detection. To achieve this, we propose private data collection with membership—any data stored on a collection is visible only to the collection members, and the hash of the data is stored on the replicated ledger. Using private data collection, Figure 5 showcases how we can achieve data privacy in the food traceability use-case. There are four private data collections, each with two members and one private data collection with

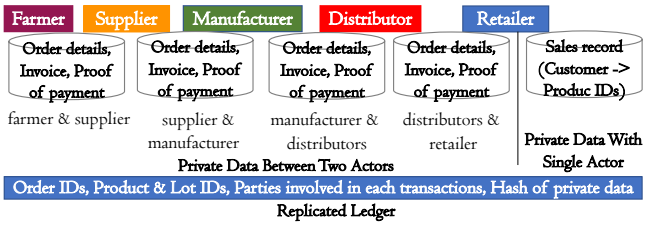


Figure 5: The private/sensitive data is visible only to the appropriate parties and the proof of data existance is stored in a replicated ledger along with necessary information needed to enable traceability.

just one member. The replicated ledger contains only the order IDs, product & lot IDs, participants involved in each transaction, and the hash of each private data stored in each collection. For each order ID, the respective order details, invoices, and payment are stored in the respective collection depending on who issued the order to whom. Thus, data privacy is achieved in the food traceability solution. Similarly, Figure 6 uses private data collection to achieve data privacy in health-care use-case. Note that the medical record can be shared with the regulator and insurance provider on an as-needed basis by copying the private data from one's hospital collection to another. As the evidence of the data is present in the replicated ledger, the receiving party can ensure that the data has not tampered. Similarly, a medical record of a patient maintained at one hospital can be copied to another on an as-needed basis. Hereafter, we refer to the data stored in the replicated ledger as public data.

Design Requirements. The following requirements should be satisfied with our design to allow a wide variety of use-cases, including the three use-cases described in §2.

R1 The smart-contract, which is executing on a blockchain node owned by a member, should be able to access both the private data collection and the public data stored on the replicated ledger. This helps the smart-contract to ensure common data formats needed for sharing data across different members and to follow the agreed business process while processing the transaction.

R2 When transactions, executed by smart-contract, modify both the public and private data, the atomicity and serializable isolation should be guaranteed.

R3 The hash of the private data stored on the replicated ledger should always be in sync with the actual private data stored in the collection.

R4 The members of a collection can be changed, i.e., a new member can be added or a old member can be removed.

R5 The sensitive data stored in a private data collection can have a time-to-live value.

4. IMPLICATION OF BLOCKCHAIN REPLICATION PROTOCOLS ON PRIVATE DATA COLLECTIONS

In this section, we consider three existing blockchain replication protocols and discuss whether the proposed private data collection can be implemented while satisfying the design requirements specified in §3. Note that it is possible to implement requirements R4 and R5 in all three existing replication protocols, and hence it is not discussed in this section.

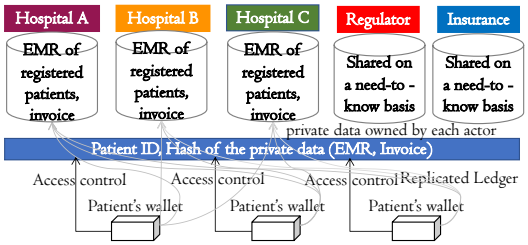


Figure 6: The private/sensitive data is maintained at the node of each individual hospital's node and the proof of data existance is stored in a replicated ledger along with necessary information needed to share data on a need-to-know basis.

4.1 Order-Execute-Commit

This replication protocol consists of the following three steps. First, the user-submitted transactions are ordered, and a block is generated through mining/consensus. Second, the created block is broadcasted to all nodes in the blockchain network. Third, on receiving a block, each node executes the transaction serially by invoking the appropriate smart-contract and commit the same. Ethereum [], Quorum [], and Hyperledger Besu [] implements order-execute-commit replication protocol. In this protocol, the transaction inputs are passed to all nodes via a block. All nodes execute the smart-contract using the inputs. Given that transaction inputs go to all nodes, we could pass the private data to members' node via a separate mechanism and only pass the hashed state in the transaction inputs. However, with this solution, it is not possible to support requirements R1, R2, and R3 listed in §3. The requirement R1 cannot be satisfied because the smart-contract is executed on all nodes, i.e., even on the nodes which do not have access to the required private data collection. The requirement R2 cannot be satisfied because the user cannot pass the pointer to the private data key that needs to be modified in the transaction input as it would reveal some information about the private data. The requirement R3 cannot be satisfied as we need to store the private data separately before or after storing the hash on the replicated ledger using the smart-contract. Given these drawbacks, we decide not to pick the blockchain platforms which support only the order-execute-commit replication protocol.

4.2 Execute-Order-In-Parallel & Commit

In this replication protocol, all transactions execute in parallel on all nodes without prior knowledge of ordering while ordering happens parallelly through a consensus service or mining. Once the block is created and delivered to all nodes, the executed transactions would get committed/aborted depending on the serializability check. If a node never received a transaction that is present in the block, it would execute the transaction and commit. The Blockchain-RDBMS [] implements execute-order-in-parallel & commit replication protocol. Similar to the order-execute-commit protocol, we cannot satisfy requirements R1, R2, and R3 for the same reason specified in the §4.1.

4.3 Simulate-Order-Validate-Commit.

This replication protocol consists of the following four steps. First, the transaction is simulated on a sub-set of nodes depending on the endorsement policy defined for a smart-contract. As a result of transaction simulation, the read-write set is collected. Second, the endorsed simulated result, which includes read-write set and endorsements, is sent to the consensus service for transaction ordering and block creation. Once the block is created, it is delivered to all nodes in the blockchain network. Thrid, on receiving the block, each node verifies whether the transaction has collected enough endorsements as per the policy defined for the invoked smart-contract and executes Optimistic Concurrency Control [] using the read-write set to perform serializability check. Finally, only valid serializable transactions are committed. Hyperledger Fabric [] implements simulate-order-validate-commit replication protocol.

With this replication protocol, we can implement private data collection while satisfying all five requirements mentioned in §3. To satisfy requirement R1, we can pass the private data in the transaction inputs itself and simulate the transaction on nodes which are the members of the private data collection. To satisfy requirements R2 and R3, we can make the node to compute and store only the hash of private data in the read-write set which goes to all nodes to get stored in the replicated ledger. During transaction simulation, we can store the private data in a temporary store and fetch it while processing the read-write set during the commit time.

5. DESIGN OF PRIVATE DATA COLLECTIONS

5.1 Transaction Flow

5.1.1 *Transaction Simulation*

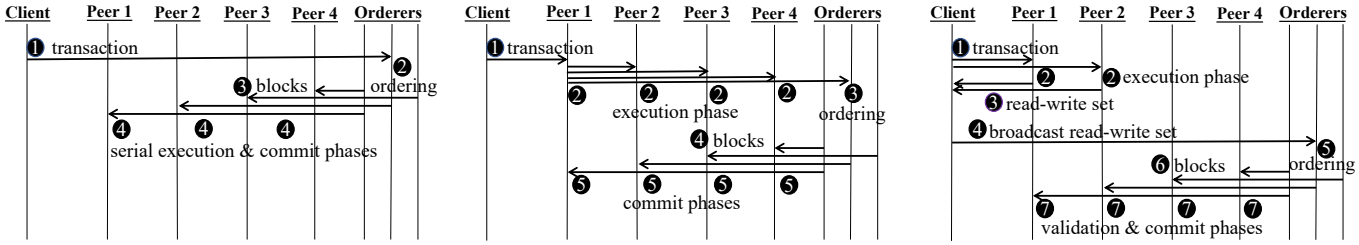
Transient Data in Transaction Proposal. Each transaction in the blockchain contains the client's input to the smart contract computation, for auditing. However, client input to smart contract computation might be confidential and thus shouldn't appear in the blockchain's blocks. Otherwise, non-authorized observers could then easily obtain sensitive information such as encryption keys, raw biometric data, social security numbers, and more.

Encrypting this secret information at the client side and decrypting it in the smart contract, is problematic:

- Using symmetric encryption induces either chicken-and-egg problem or the need to send these keys separately, complicating the interaction between clients and peers.
- It is tempting to use asymmetric encryption and have the client encrypt the secrets using the public key of the peers. This requires the client to encrypt the data separately for each peer that is involved in the smart contract execution, however requires infrastructure for public key discovery.

Instead, the client passes the secret information to the peer in a different data section, denoted as *transient data* than the public input to the smart contract computation which is omitted from the transaction that ends up in the blockchain.

In order to preserve some minimal level of auditing capabilities, the peer includes the hash of all data sent from the client (including the transient data) in the input to the signature over the smart contract computation results.



(a) Order-Execute-Commit (b) Execute-Order-In-Parallel & Commit (c) Execute-Order-Validate-Commit

Figure 7: Transaction model in Existing Blockchain Replication Protocol.

This allows a client to prove to an auditor that the computation was either done correctly or incorrectly by knowing the transient data.

Hashed, and Private Read Write Set.
Owner: Manish

While simulating a transaction, in addition to the read-write set for the public data, we generate another read-write set for the hashes of the private data referred to as *hashed read-write set*. Further, we produce a separate write-set for each private data collection that the transaction modifies. We refer to this write-set as *private write-set*. A private write-set contains only writes for the private data for a specific collection.

During simulation of a transaction, for each key in a private data collection that the transaction reads, we add a tuple <collection name, hash of the key, version of the hash of the key> to the hashed read-set. Further, for each key in a private data collection that the transaction creates or modifies, we add a tuple <collection name, hash of the key, hash of the new value> to the hashed write-set and in addition, we add a tuple <pre-images of the private key, pre-images of the new value> to the collection specific private write-set.

Thus, we generate one public read-write set, one hashed read-write set and one or more private write-sets for a simulation of a transaction.

Like the public read-write set, we package the hashed read-write set with the transaction data inside the block. However, we share each private write-sets associated with the transaction with only blockchain nodes that belong to the organizations that are member of the corresponding private data collection.

Transient Store.
Owners: Artem/Yacov

Private Data Dissemination. One can think of the data corresponding to a transaction as being split to two parts: (a) a public part which ends up being in the block and is comprised of the public read-write set and the hashed read-write set, and (b) a private part which consists of the pre-images to the hashed writes that does not end up inside the block.

Prior to the transaction being sent to be included in a block, the pre-images are being locally stored on the peers that take part in executing the transaction. ≪Yacov: this breaks abstraction, but I don't see a way out of this≫ Clearly, peers that did not take part in executing the transaction need to obtain the pre-images in some way for the private data to be available to the application.

While relying on fetching data from peers that possess the pre-images upon demand is possible, it is not a fault tolerant approach since if the peers that possess the pre-images are unreachable or crash, the private data is forever lost and as a result, user experience is impaired.

To that end, in addition to storing the pre-images locally, each peer also disseminates them to a random set of remote peers. The set is selected by applying the policy that defines the collection membership on the set of peers that are considered alive at that time, filtering out non-eligible peers and then taking a random subset of the result.
The peers that receive the pre-images, store them locally and then return acknowledgments without which the transaction is not sent to be included in a block. This is done to ensure the pre-images will be available at a later point in time in spite of faults, but can also be tuned and relaxed upon demand.

5.1.2 *Transaction Validation and Commit*

Policy Validation.
Owners: Artem/Yacov

Fetching of Private Data.

Despite hash pre-images can be disseminated to target peers before they receive the block that contains the corresponding hashes, a robust solution is needed with withstand network partitions and peer crashes which prevent the hash pre-images from reaching all the required peers. Furthermore, if new peers are added to the Blockchain, they can only obtain the required pre-images by pulling them from peers that posses them.

To that end, prior to committing a block to the ledger, a peer iterates through all hashed key writes in all transactions of the Block and analyzes whether it needs to pull private data for each hashes key write. A hashed key write is considered as a viable candidate for pulling pre-images if:

- The transaction passed validation of endorsement policies ≪Yacov: Are we into abstracting out EPs? If yes then how should I phrase this?≫. This is needed in order to prevent a denial-of-service attack in which a malicious party creates transactions with hashed writes for which there are no pre-image in any peer in the network. Peers would try to pull these transactions and fail, hence considerably slowing down throughput.

- Evaluation of the collection policy on the peer's identity returns "true" ≪Yacov: Once we define collection policies more abstractly we need to change this as well≫: this is required since peers that receive pre-image requests for a certain hashed write also evaluate the same collection policy on the requester peer's identity in order to determine if that peer is part of the collection the hashed write is associated with.

- Whether the peer is missing the pre-image from its transient store, since by definition this means the pre-

image hasn't been disseminated to the peer before the commit of the block.

- ≪Yacov: Should we mention purged private data?≫

Once the peer decides it needs to pull a hashed write, it needs to select potential peers which might have the pre-image it needs, and send them requests for the digests. Upon reception of responses containing pre-images, the peer validates that the hashes on the pre-images received matches the hashed write in the block. It follows from the collision resistance property of the cryptographic hash function that if the responding peer sent a pre-image of a hashed write, then with overwhelming probability, the pre-image is identical to all pre-images in all peers in the network, hence consistency is preserved.

When sending requests for pre-images, a requester peer doesn't know up-front whether the responder peer has the pre-image, and every such "wrong guess" results in a retry which delays the block commit and thus hinders performance.

For this reason, the peers prioritize requesting a pre-image from the peers which endorsed the transaction ≪Yacov: This is very Fabric specific, so we might want to remove this?≫ from which the hashed write originated. This is possible because peers constantly maintain a membership view of alive peers by periodically disseminating heartbeats.

Serializability Validation.
Owner: Manish

In order to maintain serializability of the transactions, we validate each transaction in the block, *logically* in the order in which they apper in the block. In order to perform validation for a transaction, we verify that the version of the each of key hashes in the hashed read-set are the same as in the committed state as of the immidiate preceding transaction. If the transaction is found to be valid, then we apply to the committed state the write-set portion of the hashed read-write-set and then we proceed with the validation of the next transaction in the block.

This approach allows for the peers across organizations to perform validation of the transactions in a deterministic manner without having to access to the actual private data. As a result, this approach allows for modifying in a single transaction, the public data and the private data from more than one collections. However, this approach comes with a limitation, which is, unlike public data, we can only support the point queries on the private data with a serializability guarantee. The support of complex queries would work only where applications can tolerate with repeatable read semantics for the transaction isolation.

Private Data Store.
Owner: Manish

We prserve each of the private write-set associated with a collection within a transaction in a separate store that we refer to as *private data store*. Since each organization may be eligible to receive the private write-sets for a different set of collections in a transaction, unlike block store, the data stored in the private data store on one peer may differ from the data stored into another peer with in a network. The primary motivation for the private data store is to maintain all the versions of private data for the provenance related queries. The private data store also serves the request from other organizations for the private write-sets associated with a transaction. However, the private data store serves the write-sets of only those collections in a transaction for which the requesting organization is a member.

As noted before, at the time of committing a block on a peer, some of the private write-sets may be not be available corresponding to the transactions included in the block. There are two primary reasons for non availability of the private write-sets for a collection at the time of committing a block; 1) the organization is not eligible for one or more collections modified in a transaction and 2) the committing peer could not successfully access to the private write-sets. In the private data store, in addition to persisting the private write-sets, we also maintain this information about the non-availability of the private write-sets along with a notion that specifies one of the above two mentioned reasons. This information is used for attempting to retrieve the missing private write-sets in the future under two scenarios; 1) the peer periodically tries to fetch the private write-sets for which the peer is eligible but the peer could not retrieve this data from other peers at the time of commit and 2) if the peer becomes eligible for a collection in the future. We discuss these scenarios in more detail in the latter sub-sections.

State DB.
Owner: Manish

5.2 Collection Membership Change and Reconciliation

Owner: Senthil

5.3 Right to Forget

Owner: Manish

5.4 Failure and Recovery

Owner: Senthil

5.5 Fork Detection and Ledger Rollback

Owner: Senthil

6. IMPLEMENTATION

using Hyperledger Fabric

7. EVALUATION

Owner: Senthil

8. DISCUSSION AND FUTURE WORK

8.1 Other Supported Features and Use-Cases

8.1.1 Collection ACL

(1) Members Only Read-Write.
(2) Members Only Write and All Can Read.
(3) Members Only Read and All Can Write.
(4) All Can Read-Write.

8.1.2 Moving Data Between Collections

some text ...

9. RELATED WORK

(1) Fabric Channel. Hyperledger Fabric achieves data segregation through multi-tenancy by allowing nodes to participate in a plurality of independent Blockchain protocol

instances. Each such a distributed Blockchain protocol instance is called a channel.

While each channel induces its own ledger, nodes share their local storage and compute resources among the different channels they participate in.

Being independent ledgers allows channels to provide perfect privacy, as no data is shared with ineligible organizations if they are not part of a channel. However, this data segregation is extremely coarse grained and the lack of total order across different channels makes it non trivial to use for all use cases that require hiding only part of the data of a transaction. Additionally, channels come with a significant performance overhead, as all protocol messages, in-memory data structures, and database instances are now duplicated.

(2) Quorum [?] Private Transaction. Quorum is a permissioned Blockchain based on Ethereum. By using classical consensus algorithms such as Raft and Istanbul-BFT, it achieves finality and high throughput which are desired qualities for enterprise settings.

Quorum achieves privacy through sharding the state database. Private transactions are applied on namespaces which are disjoint from the public namespaces, thus unlike Hyperledger Fabric, there is no interoperability between private information and public information in the same transaction.

Quorum private transactions are implemented by first generating a symmetric key and encrypting it with public keys of each party that the transaction is designed to. Then, the encryptions of the symmetric key, and the transaction payload encrypted with the symmetric key is sent to the relevant parties. Finally, a private transaction is generated with only the hash of the encrypted payload, and is appended to the Blockchain. All nodes not included in the *privateFor* field ignore the transaction, while those that are – decrypt the symmetric key with their public key, and finally decrypt the private payload and commit the transaction.

(3) Encryption of Data. ≪Yacov: I don't think there is usage of encryption only in Blockchains, at least not that I could find. I think we should remove this part altogether.≫

(4) Zero Knowledge Proof. Proofs that yield nothing more than their validity were introduced by Goldwasser, Micali and Rackoff in 1985 [?]. Since then, significant improvements have been made, namely making the proofs non-interactive and efficient [?]. Employing zero knowledge proofs in distributed protocols enforces honest behavior on dishonest parties, allowing hiding confidential data with commitments or encryption and still retaining correctness.

Utilizing zero knowledge proofs in Blockchain were first proposed by Adam Back in 2013 [?] and incorporated into Blockchain technology by Maxwell [?] which used Pedersen commitments [?] to hide the transaction amounts, and zero knowledge proofs to ensure the sum of input coins is not less than the sum of output coins.

Later on, Monero [?] enhanced the former technique by adding ring signatures to hide the transactor among a set of random transactors, as well as hiding the target address of a coin transfer.

The biggest downside of the techniques used in Monero is that the transaction size grows linearly in the anonymity set. The authors of Zerocash [?] proposed to use a succinct zero knowledge proof system called zkSNARKs [?], however it relies on trusted setup, and although the verification is very fast, the proving time is significant.

Privacy preserving smart contracts with arbitrary logic that use zero knowledge proofs were proposed in Hawk [?] which also utilizes zkSNARKs, thus relies on a trusted setup. Zether [?] eliminates the trusted setup requirement by utilizing a novel zero knowledge scheme called Bulletproofs, and is implemented as an Ethereum smart contract for payment.

10. CONCLUSION

some text ...